

// CLASSIFICATION: INTERNAL BUILD SPEC
// DISTRIBUTION: SINGLE USER — LOCAL BUILD ONLY

TALKDIPLOMACY LIVE

v1 TECHNICAL BUILD MANUAL

Cross-stream global event correlation engine · live map · causal storyline system

REVISION	1.2 — repo setup added
STATUS	ACTIVE — pre-build
REPOSITORY	GlobeGrid (GitHub) — see Section 14 for license/gitignore
TARGET AGENT	Claude Fable 5 (autonomous, phase-scoped execution)
OWNER SCOPE	Single user, local machine, no external distribution
ENVIRONMENT	Windows, native, no containers
CORE DIFFERENTIATOR	Cross-stream correlation + long-horizon fact chain

Purpose: this document is a precise build specification, written to be executed by an AI coding agent in one or a small number of large autonomous passes. It is not a pitch document and has no external audience. Every decision below is final for v1 unless explicitly marked OPEN or DEFAULT (tunable).

SECTION 01 // SYSTEM OVERVIEW

System overview

TalkDiplomacy Live is a real-time global event intelligence system. It ingests news, structured event data, disaster data, market data, and social signal; extracts and permanently stores structured facts from every item; detects when events across different streams and different points in time are actually connected; and generates human-readable causal storylines explaining what happened, why, and what it's connected to. It is presented through a live feed, an interactive world map, and individual story pages, with a tiered graphics system so the most advanced visual experience runs on the owner's machine now, while lighter tiers remain available for future non-local use.

The defining capability is **cross-stream correlation**: identifying that a market move, a news story, and a social signal spike are describing the same underlying real-world event — and that a new event may be causally connected to something that happened months earlier via the permanent fact chain. This is not a news aggregator. It is a persistent, growing knowledge graph with a presentation layer.

1.1 Core build principles

- Single user, single machine, no accounts, no login flow.
- Every feature discussed in planning ships in v1. Nothing is deferred unless explicitly marked so.
- Every displayed fact links back to its original source, with source name and outbound link always visible — enforced at the schema level (Section 6.8).
- The correlation and causal-storyline engine is the centerpiece and receives the most implementation care.
- Facts extracted from articles are never deleted; they accumulate into a permanent chain used for long-horizon correlation.
- All tunable numeric thresholds live in one config file (Section 7.2), never hardcoded inline.

1.2 Non-goals for v1

- No user accounts, permissions, or multi-tenant support.
- No payment, subscription, or licensing logic.
- No mobile app — responsive web only, optimized for the owner's desktop first.
- No production deployment, load balancing, or horizontal scaling. Runs as local processes.
- No containerization (Docker) — see Section 3 for rationale, and Appendix A for how this changes later.

HARD REQUIREMENT: no content is ever rendered to the UI without a resolvable, visible source citation. This is enforced at the database schema level, not just the frontend — see Section 6.8.

SECTION 02 // ARCHITECTURE

Architecture and data flow

The system is a linear seven-stage pipeline with one feedback structure: the fact chain feeds forward into every future correlation pass, not just the current ingestion window.

2.1 Pipeline stages

- **Stage 1 — Ingestion.** Scheduled jobs pull from all registered sources (Section 4) on a per-source interval and write raw items to `raw_items`, unmodified.
- **Stage 2 — Extraction.** Each raw item is parsed into a normalized `events` record, and a structured fact set is pulled into `extracted_facts`.
- **Stage 3 — Embedding.** Every extracted fact and event gets a vector embedding computed from its normalized description, not raw text.
- **Stage 4 — Correlation.** A similarity + time-window pass groups embeddings into candidate clusters, checked against the full historical fact chain, not just the current window.
- **Stage 5 — Causal linking.** Each correlated cluster is passed to an LLM call producing a structured causal narrative with a confidence score, stored as a `stories` record.
- **Stage 6 — Serving.** The FastAPI layer exposes stories, events, and the instability index to the frontend (contract in Section 8).
- **Stage 7 — Presentation.** React frontend renders the live feed, map, and story pages at the graphics tier appropriate to the device (Section 11).

2.2 The fact chain (long-horizon correlation)

Unlike a standard clustering system that only compares items within a short time window, this system keeps every extracted fact permanently in `extracted_facts`, embedded and indexed. Stage 4 always checks new events against the full historical chain (bounded by a similarity threshold, not a time window) so a new event can be linked to something that happened months earlier. This is the structural difference between this system and a point-in-time aggregator.

2.3 Failure isolation model

No single stage failure may halt the pipeline. Each ingestion job, each correlation pass, and each causal-link call is wrapped independently with its own error boundary (detail in Section 10). A slow or dead source degrades gracefully; it never blocks unrelated sources or the serving layer.

SECTION 03 // TECH STACK

Tech stack (locked)

3.1 Locked decisions

Field	Type	Description
Backend language	Python	FastAPI framework, Python 3.12+
Database	PostgreSQL 16	with PostGIS extension, installed natively on Windows
Similarity search	pgvector (preferred) / Python fallback	see 3.2 install notes
Frontend	React + Vite	no Next.js — no SEO/SSR requirement for a local single-user app
Embeddings	sentence-transformers	all-MiniLM-L6-v2, 384-dim, runs on CPU, no external API needed
Causal-link LLM	Claude API	claude-sonnet-5 by default; model name is an env var, not hardcoded
Map rendering — Tier 1	three.js / WebGL2	custom globe renderer, see Section 11
Map rendering — Tier 2	Mapbox GL JS or Leaflet	flat 2D fallback
Scheduling	APScheduler	in-process; no Celery/Redis needed at single-user scale
Realtime transport	native WebSocket (FastAPI)	no external pub/sub broker needed at this scale
OS / environment	Windows, native	no Docker, no WSL; install steps target native Windows paths

Rationale for Postgres over SQLite despite single-user scale: PostGIS spatial queries (radius search, geographic clustering) and vector similarity search at the database level materially simplify Stage 4. This is worth the extra install step.

3.2 Windows install sequence

- Install PostgreSQL 16 via the official EDB Windows installer (postgresql.org/download/windows).
- During install, use StackBuilder (bundled) to add the PostGIS extension — the standard native-Windows path, avoids manual compilation.
- After creating the target database, run `CREATE EXTENSION postgis;`.
- Install `pgvector` for embedding similarity search — prebuilt Windows binaries are available for supported PG versions; if unavailable, fall back to storing embeddings as float arrays and computing cosine similarity in Python at query time for v1 (flagged DEFAULT — revisit once real data volume is known).
- Python: use a virtual environment (`python -m venv .venv`), never a global install.

- Node: use current LTS for the Vite frontend build.
- Verify install: `psql -U postgres -c "SELECT postgis_full_version();"` should return a version string with no error.

SECTION 04 // DATA SOURCES

Data sources (locked list for v1)

All sources below are free and require no paid key at v1 usage levels. Every source has an entry in [sources](#) (Section 6.1) used to render attribution.

4.1 News (RSS)

RSS feeds

Field	Type	Description
BBC World	RSS	https://feeds.bbc.co.uk/news/world/rss.xml
Al Jazeera	RSS	https://www.aljazeera.com/xml/rss/all.xml
NYT World	RSS	https://rss.nytimes.com/services/xml/rss/nyt/World.xml
Washington Post	RSS	https://feeds.washingtonpost.com/rss/national
CNN World	RSS	http://rss.cnn.com/rss/edition_world.rss
NPR	RSS	https://feeds.npr.org/1001/rss.xml
Reuters (via Google News)	RSS	https://news.google.com/rss/search?q=when:24h+allinurl:reuters.com — Reuters retired direct RSS in 2020; this is the standard workaround

4.2 Structured global event data

GDELT

Field	Type	Description
GDELT DOC 2.0 API	REST, free, no key	api.gdeltproject.org/api/v2/doc/doc — full-text search across global news, rolling 3-month window
GDELT Cloud	REST, structured	docs.gdeltcloud.com — pre-clustered Stories, coded Events, linked Entities; updates hourly; strongest coverage from March 2026 onward

Treat GDELT Cloud as a primary source, not supplementary — it performs a portion of the clustering work upstream and can seed the correlation engine directly.

4.3 Physical world events

Disaster / physical data

Field	Type	Description
USGS Earthquakes	GeoJSON, free, no key	earthquake.usgs.gov feeds — real-time, updated continuously

4.4 Markets

Market data

Field	Type	Description
Alpha Vantage	REST, free tier, key required	Price/volume data for correlating market moves with news/social events; free tier rate limits apply, cache aggressively

4.5 Social

Social data

Field	Type	Description
Reddit API	REST, free, OAuth app key	Most realistic free social source at this scale; X/Twitter excluded (prohibitive API pricing); Telegram excluded from v1 (fragile per-channel scraping)

Every record derived from a source carries a `source_id` foreign key. Every story page and feed item displays the originating outlet name and a direct outbound link, unobscured, above the fold of that item.

SECTION 05 // FEATURE SPECIFICATIONS

Feature specifications

5.1 Live feed

WHAT IT DOES — A continuously updating stream of recent stories, ranked by recency and cluster significance.

DATA IN / OUT — Reads `stories`; pushed to frontend via WebSocket on new insert; falls back to 15-second polling if the connection drops.

- Per-source ingestion interval (DEFAULT, config value): RSS every 300s, GDELT Cloud every 3600s (matches its own cadence), USGS every 120s, market data every 300s during market hours, Reddit every 600s.
- Feed shows story clusters, not raw articles — one card per correlated event, with a source-count badge.
- Feed never blocks on a single slow source; each ingestion job runs independently (Section 2.3, Section 10).

5.2 World map

WHAT IT DOES — Every event plotted geographically, showing where things are happening, not just a list.

DATA IN / OUT — Reads `events.location`; renders via Tier 1/2/3 renderer per Section 11.

- Events without a resolvable location are excluded from the map view but remain in the feed.
- Pin density above a threshold (DEFAULT: 15 pins within 300km) collapses into a cluster marker showing count; clicking expands.
- Map supports filtering by category and by instability contribution (Section 5.6).

5.3 Story pages

WHAT IT DOES — One page per correlated event cluster: full source list, timeline, and the causal explanation.

DATA IN / OUT — Reads one `stories` row plus linked `events`, `extracted_facts`, and `causal_links` rows.

- Layout: headline, one-paragraph AI-generated summary, confidence indicator, timeline of which source lit up when, full source list with outbound links, and a 'connected history' panel surfacing linked past facts from the chain.
- The 'connected history' panel is the surfaced form of the fact chain — the element that differentiates a story page from a normal aggregator article page.

5.4 Cross-stream correlation engine

WHAT IT DOES — Detects when a market move, a news story, and a social spike are the same underlying event — the system's core differentiator.

DATA IN / OUT — Reads embeddings from `extracted_facts` and `events`; writes cluster assignments to `stories` and `story_members`.

- Step 1: reduce every incoming item to a normalized event description (entities + location + action, not raw headline text) before embedding — this is what makes cross-source matching work, since raw text differs wildly by source for the same event.
- Step 2: embed with all-MiniLM-L6-v2 (384-dim).
- Step 3: compare against (a) other items in the current ingestion window under a tight time gate, and (b) the full historical fact chain via cosine similarity only, no time gate, for long-horizon links.
- Step 4 (DEFAULT thresholds, config-driven, see 7.2): 0.78 cosine similarity for same-window clustering, 0.85 for cross-time historical linking — the higher bar reflects that false positives across large time gaps are more damaging to trust than same-day misses. Tune once real data is flowing.
- Step 5: geographic overlap is a secondary signal, not a hard gate — two events in different locations can still be one story (e.g. a policy announced in one capital affecting markets elsewhere).

5.5 Causal storylines

WHAT IT DOES — Explains what likely caused an event and what it's likely to affect, instead of just listing facts.

DATA IN / OUT — One Claude API call per newly formed or updated story cluster. Input: all member facts/events with timestamps. Output: structured JSON (cause, affected, consequences, confidence). Full prompt template in Section 9.

- Prompt explicitly instructs the model to state uncertainty rather than false certainty, and to use timestamp order as a first-pass signal for causal direction.
- Output is cached on the story record; regenerated only when new members are added to the cluster, never on every page view.
- Confidence level is always displayed to the reader alongside the narrative — never presented as fact without it.

5.6 Instability index

WHAT IT DOES — One composite number reflecting how tense/chaotic the world is right now.

DATA IN / OUT — Computed on a schedule (DEFAULT: every 900s) from active stories in a rolling 72-hour window; written to `instability_scores` with a timestamp for historical tracking.

- v1 formula: weighted sum of (event volume, average severity tag, number of distinct regions involved), normalized to 0-100. Weights are config values (Section 7.2), not hardcoded.
- Score history is retained indefinitely and rendered as a trend line on the homepage.

5.7 Bias / blindspot view

WHAT IT DOES — Shows how the same story is covered differently across outlets.

DATA IN / OUT — Groups a story's member articles by `source_id`; joins against a static outlet-leaning reference table.

- v1 uses a small manually-curated leaning table for the sources in Section 4.1 (left/center/right label) rather than an external dataset dependency — keeps v1 self-contained.
- Displayed as a side-by-side: outlet name, leaning label, and their headline for this story.

5.8 Multi-language support

WHAT IT DOES — Non-English content is usable without a separate pipeline per language.

DATA IN / OUT — Translation happens at display time, not ingestion time — original text and language are always preserved.

- GDELT's live-translated 65-language coverage is used as-is where available.
- For v1, translation-on-display can call the Claude API for short summary translation; full-article translation is out of scope (link to original suffices).

5.9 Fact chain / long-horizon memory

WHAT IT DOES — Every extracted fact persists permanently and feeds every future correlation pass, not just the current window.

DATA IN / OUT — Writes to `extracted_facts` on every processed item; never deleted, never expired.

- This table is the system's memory. Its embedding index must support efficient similarity search at scale (pgvector or equivalent) since it grows unbounded.
- Each fact stores its own embedding, source reference, timestamp, and any story clusters it has ever been linked to (many-to-many via `story_members`).

5.10 Ingestion resilience

WHAT IT DOES — One dead source never breaks the whole system.

DATA IN / OUT — Each ingestion job wrapped independently; failures logged, not raised to the scheduler.

- Each source has a health status (ok / degraded / down) tracked in `sources`, visible in a system-status panel in the UI.
- On failure: exponential backoff up to a max interval, never a hard crash of the scheduler process. Full policy in Section 10.

SECTION 06 // DATA SCHEMA

Data schema (PostgreSQL + PostGIS)

All tables live in a single database, e.g. `talkdiplomacy_live`. Geospatial columns use PostGIS `geography(Point,4326)`. Embedding columns use `vector(384)` via pgvector where available.

6.1 sources

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	Primary key
<code>name</code>	<code>text</code>	Display name, e.g. 'BBC World'
<code>type</code>	<code>text</code>	rss gdelt usgs market reddit
<code>url</code>	<code>text</code>	Feed or API endpoint
<code>leaning</code>	<code>text</code>	left center right n/a — used by Section 5.7
<code>poll_interval_seconds</code>	<code>int</code>	How often this source is fetched
<code>health_status</code>	<code>text</code>	ok degraded down
<code>last_fetched_at</code>	<code>timestamp t z</code>	Last successful fetch
<code>last_error</code>	<code>text</code>	Nullable; last error message for diagnostics

6.2 raw_items

Field	Type	Description
<code>id</code>	<code>uuid PK</code>	Primary key
<code>source_id</code>	<code>uuid FK</code>	References sources.id
<code>raw_content</code>	<code>jsonb</code>	Unmodified payload as received, for reprocessing/debugging
<code>fetches_at</code>	<code>timestamp t z</code>	Ingestion timestamp
<code>processed</code>	<code>boolean</code>	Whether Stage 2 extraction has run on this item
<code>processing_error</code>	<code>text</code>	Nullable; set if Stage 2 failed on this item

6.3 events

Field	Type	Description
id	uuid PK	Primary key
raw_item_id	uuid FK	References raw_items.id
title	text	Normalized event title
description	text	Normalized 'what happened' description used for embedding
location	geography(Point,4326)	Geocoded location, nullable
location_name	text	Human-readable place name
category	text	geopolitics finance disaster conflict other
severity	int	1-5 scale, used by instability index
occurred_at	timestampz	Best-estimate event time
embedding	vector(384)	all-MiniLM-L6-v2 embedding

6.4 extracted_facts

Field	Type	Description
id	uuid PK	Primary key
event_id	uuid FK	References events.id, nullable if fact predates event linkage
source_id	uuid FK	References sources.id — required, enforces attribution
who	text	Entities involved
what	text	Normalized action/fact
where	text	Location text
when_occurred	timestampz	Fact timestamp
embedding	vector(384)	Used for long-horizon similarity search
created_at	timestampz	Row insert time, never modified

6.5 stories

Field	Type	Description
id	uuid PK	Primary key
headline	text	AI-generated cluster headline
summary	text	AI-generated one-paragraph summary
causal_narrative	jsonb	Structured output from Section 5.5: cause, affected, consequences
confidence	text	high medium low
first_seen_at	timestamp z	Earliest member event time
last_updated_at	timestamp z	Last time a new member was added

6.6 story_members

Field	Type	Description
story_id	uuid FK	References stories.id
event_id	uuid FK	References events.id, nullable
fact_id	uuid FK	References extracted_facts.id, nullable
linked_via	text	same_window historical_chain — which correlation path added this member
linked_at	timestamp z	When this member joined the cluster

6.7 instability_scores

Field	Type	Description
id	uuid PK	Primary key
score	numeric(5, 2)	0-100 composite score
computed_at	timestamp z	Timestamp of this reading
component_breakdown	jsonb	Volume/severity/spread sub-scores for transparency

ATTRIBUTION IS SCHEMA-ENFORCED: every table that can appear on a rendered page (events, extracted_facts, stories via story_members) traces back to a non-nullable source_id, either directly or via its parent row. No content is ever displayed without a resolvable source.

SECTION 07 // CONFIGURATION

Configuration and environment

All tunables live in two places: environment variables for secrets/connection info, and a single config file for thresholds/intervals/weights. Nothing tunable is hardcoded inline in application code.

7.1 .env.example

```
DATABASE_URL=postgresql://postgres:PASSWORD@localhost:5432/talkdiplomacy_live
CLAUDE_API_KEY=sk-ant-...
CLAUDE_MODEL=claude-sonnet-5
ALPHAVANTAGE_API_KEY=...
REDDIT_CLIENT_ID=...
REDDIT_CLIENT_SECRET=...
REDDIT_USER_AGENT=talkdiplomacy-live/1.0 (single-user local build)
EMBEDDING_MODEL_NAME=all-MiniLM-L6-v2
API_PORT=8000
WS_PORT=8000
LOG_LEVEL=INFO
LOG_DIR=./logs
```

7.2 config.yaml — tunable defaults

```
correlation:
  same_window_similarity_threshold: 0.78
  historical_similarity_threshold: 0.85
  same_window_max_gap_hours: 72
  geo_overlap_radius_km: 300
```

```
instability:
  weight_volume: 0.4
  weight_severity: 0.4
  weight_spread: 0.2
  recompute_interval_seconds: 900
  rolling_window_hours: 72
```

```
ingestion_intervals_seconds:
  rss: 300
  gdelt_cloud: 3600
  usgs: 120
  market: 300
  reddit: 600
```

```
map:
  cluster_pin_threshold: 15
  cluster_radius_km: 300
```

```
resilience:
  max_backoff_seconds: 1800
  backoff_multiplier: 2.0
```

All values above are DEFAULT starting points, not final facts — they are genuinely un-knowable with precision before real data is flowing, and are deliberately isolated in one file so they can be tuned without touching

application logic.

SECTION 08 // API CONTRACT

API and WebSocket contract

8.1 REST endpoints

Field	Type	Description
GET /api/stories	since, limit, category (query)	List recent story clusters, newest first
GET /api/stories/{id}	id (path)	Full story detail: members, causal_narrative, sources
GET /api/events	bbox, category, since (query)	Events within a map bounding box
GET /api/instability	range (query, default 72h)	Latest score + historical trend
GET /api/sources/status	none	Health status of every registered source
WS /ws/feed	none — persistent connection	Real-time push of new/updated stories and instability updates

8.2 WebSocket message envelope

```
{
  "type": "story_created" | "story_updated" | "instability_updated",
  "payload": { ... },
  "timestamp": "2026-07-02T14:31:00Z"
}
```

Frontend reconnects with exponential backoff on drop (matches the resilience policy in 7.2) and falls back to 15-second REST polling of `GET /api/stories?since=...` if the socket cannot be reestablished within 60 seconds.

SECTION 09 // CAUSAL LINKER

Causal linker prompt template

One call per newly formed or updated story cluster (Section 5.5). The model receives every member fact/event with timestamps and must return strict JSON — no prose outside the JSON object.

System prompt

You are a geopolitical and global-events analyst. You will be given a cluster of correlated facts, each with a timestamp and source. Determine whether they describe a connected sequence of cause and effect.

Return ONLY valid JSON matching this shape:

```
{
  "cause": string,
  "affected": string[],
  "consequences": string[],
  "confidence": "high" | "medium" | "low"
}
```

Rules:

- Never state certainty the evidence does not support.
- Use timestamp order as a first signal for causal direction, but only state a direction when the evidence actually supports it.
- If the cluster does not support a clear causal story, set confidence to "low" and describe the connection as correlational, not causal.
- Do not invent facts not present in the input.

User payload (example shape)

```
{
  "cluster_facts": [
    {"source": "BBC World", "timestamp": "2026-07-01T09:12:00Z",
     "who": "Central Bank of X", "what": "raised interest rates 50bps",
     "where": "Country X"},
    {"source": "Alpha Vantage", "timestamp": "2026-07-01T09:40:00Z",
     "who": "Currency X", "what": "dropped 2.1% against USD",
     "where": "Country X"},
    {"source": "Reddit r/economics", "timestamp": "2026-07-01T10:05:00Z",
     "who": "retail traders", "what": "spike in discussion volume",
     "where": "n/a"}
  ]
}
```

Output is parsed, validated against the JSON shape, and stored on `stories.causal_narrative`.

Malformed output triggers one retry; a second failure marks the story confidence as 'low' with a null narrative rather than blocking the pipeline.

SECTION 10 // OBSERVABILITY

Logging, resilience, and observability

10.1 Structured logging

- Every ingestion job run logs one structured JSON line: `source_id`, `status`, `item_count`, `duration_ms`, `error` (nullable).
- Every causal-link LLM call logs token count and latency, for cost visibility — not urgent at single-user scale, but the habit transfers directly to a future multi-user version (Appendix A).
- Logs write to a local rotating file (`LOG_DIR/app.log`, size-based rotation) in addition to console output.

10.2 Failure isolation policy

- Each ingestion job is wrapped in its own try/except; a failure updates that source's `health_status` and `last_error`, and does not propagate to the scheduler process.
- Backoff on repeated failure: multiply the poll interval by `backoff_multiplier` (default 2.0) up to `max_backoff_seconds` (default 1800), then reset on next success.
- Correlation and causal-linking failures on one cluster never block processing of other clusters in the same batch.
- A dead source is surfaced in the UI status panel (Section 5.10), never silently dropped.

SECTION 11 // GRAPHICS TIERS

Graphics tiers

Three tiers, detected on load and manually overridable. Tier 1 is built first, against a synthetic dataset matching the real schema exactly (Section 12), so the real correlation engine can be swapped in later without touching rendering code.

11.1 Tier definitions

Item	Description
Tier 1 — Full	WebGL 3D globe (three.js), animated correlation threads between linked stories, particle-density effects for event clusters. Target: 60fps on the owner's Alienware-class hardware.
Tier 2 — Standard	Flat 2D map (Mapbox GL or Leaflet), clustered pins, smooth CSS transitions, no WebGL requirement. Target: works on any modern laptop/desktop browser.
Tier 3 — Light	List/card view with a static map image or no map. Prioritizes content loading instantly. Target: under 2 seconds to first content on a mid-range phone.

11.2 Detection logic

- On load: check WebGL2 support, screen width, and Network Information API connection type.
- WebGL2 + screen width > 1024px + non-'slow-2g' connection → default Tier 1.
- WebGL1-only or mid-range signals → default Tier 2.
- No WebGL or explicit low-end signal → default Tier 3.
- A manual override control is always visible regardless of auto-detected tier — auto-detect is a default, not a lock.

11.3 Performance budgets

Budgets

Field	Type	Description
Tier 1 frame rate	target	60fps sustained with up to 500 simultaneous rendered events
Tier 1 initial load	target	under 4 seconds to interactive on target hardware
Tier 2 initial load	target	under 2.5 seconds to interactive
Tier 3 initial load	target	under 2 seconds to first content, any device

SECTION 12 // SYNTHETIC DATASET

Synthetic dataset generator

Before the real pipeline is wired in, Tier 1 graphics are built against a generated dataset matching the Section 6 schema exactly. Generated by a script, not hand-written JSON, so volume and distribution can be adjusted to stress-test rendering performance before real data exists.

12.1 Generator parameters (script: `scripts/generate_synthetic_data.py`)

- Default: 300 synthetic `events` distributed across ~40 real-world population centers with randomized jitter.
- Default: 60 synthetic `stories`, each linking 2-6 events via `story_members`.
- `causal_narrative` populated with structurally valid, clearly-labeled placeholder JSON (not real analysis).
- `instability_scores` backfilled hourly for the past 7 days, so the trend line has data to render immediately.

12.2 Example generated event row

```
{
  "id": "b3f1...-synthetic",
  "title": "Port congestion reported at major shipping hub",
  "description": "shipping delays reported, Port of X, container backlog",
  "location_name": "Port of X",
  "category": "finance",
  "severity": 3,
  "occurred_at": "2026-06-29T04:00:00Z",
  "_synthetic": true
}
```

Synthetic rows are flagged with an internal `_synthetic` marker so they can be purged in a single operation once the real pipeline (Phase 1-3) is wired in for Phase 5.

SECTION 13 // PROJECT LAYOUT

File and folder structure

```

globegrid/
  LICENSE                                GNU AGPL-3.0 (see Section 14.1)
  .gitignore                             Python template + Node additions (see Section 14.2)
  README.md
  backend/
    app/
      main.py                            FastAPI app entrypoint
      config.py                           Loads .env + config.yaml (Section 7)
    db/
      models.py                           SQLAlchemy models matching Section 6 schema
      session.py
    ingestion/
      sources/                             One module per source type: rss.py, gdelt.py, usgs.py, market.py, reddit.py
      scheduler.py                         APScheduler job registration
    processing/
      extract.py                           Stage 2: raw_item -> event / extracted_fact
      embed.py                             Stage 3: embedding generation
      correlate.py                          Stage 4: clustering logic (Section 5.4)
      causal_link.py                        Stage 5: Claude API call + prompt template (Section 9)
      instability.py                       Section 5.6 scoring job
    api/
      routes_stories.py
      routes_events.py
      routes_map.py
      routes_status.py
    websocket/
      feed_socket.py
      logging_setup.py                     Section 10 structured logging config
  requirements.txt
  .env.example
  config.yaml
  frontend/
    src/
      components/
        map/
          Tier1Globe.jsx
          Tier2Map.jsx
          Tier3List.jsx
          TierDetector.js
        feed/
        story/
      data/
        syntheticDataset.js               Mirrors Section 12 generator output for local dev
      api/
        client.js
        socket.js
      App.jsx
      vite.config.js
      package.json
    docs/
      this manual (source form)

```

scripts/	
generate_synthetic_data.py	Section 12
db_bootstrap.sql	CREATE EXTENSION postgis; CREATE EXTENSION vector; etc.
logs/	
app.log	Rotating structured log output (Section 10.1)

SECTION 14 // REPOSITORY SETUP

License and .gitignore

Repository name: [GlobeGrid](#) (GitHub). This section is the locked decision record for the repo's legal and hygiene setup — both are configured at repo creation, before Fable writes any code.

14.1 License — GNU AGPL-3.0

GlobeGrid is licensed under the GNU Affero General Public License v3.0. Selected via GitHub's license dropdown at repo creation ("GNU Affero General Public License v3.0"), which generates the correct [LICENSE](#) file automatically — no manual drafting required.

- **Why AGPL over plain MIT/GPL:** AGPL's network-use clause closes the SaaS loophole in standard GPL — if a third party runs a modified GlobeGrid as a hosted service without redistributing the binary, they are still required to make their modified source available to users of that service. This matches the goal stated during planning: free for mass use, but not silently cloneable into someone else's closed paid product.
- **Scope:** AGPL governs code written for this project only. Third-party dependencies (FastAPI, React, three.js, sentence-transformers, etc.) retain their own licenses — all are permissively licensed (MIT/BSD/Apache-family) and compatible with AGPL as dependencies.
- **Precedent:** this mirrors the licensing choice made by comparable open-source global-intelligence-dashboard projects in this space, for the same reason — protecting a free public tool from uncompensated closed-source forks.

14.2 .gitignore — Python template + Node additions

GitHub's repo-creation flow allows exactly one starter template; **Python** was selected (matches the FastAPI backend). The Node-relevant lines below are appended manually immediately after repo creation, before the first commit, since the frontend is a separate Vite/React app not covered by the Python template.

```
# --- from GitHub's Python template (selected at repo creation) ---
__pycache__/
*.py[cod]
.venv/
venv/
*.egg-info/
.pytest_cache/
.mypy_cache/

# --- appended manually: Node / frontend (Vite + React) ---
node_modules/
dist/
.vite/

# --- appended manually: project-specific ---
.env
.env.local
config.local.yaml
logs/*.log
*.db
```

```
*.sqlite3
scripts/__pycache__/

# --- appended manually: OS / editor noise ---
.DS_Store
Thumbs.db
.vscode/
.idea/
```

The .env line is the one that actually matters — it holds the Claude API key, Reddit OAuth credentials, and Alpha Vantage key. It must be in place before the first commit, not added retroactively.

SECTION 15 // BUILD SEQUENCE

Build sequence for Fable

Recommended execution: phase-scoped prompts, not one single shot and not feature-by-feature micromanagement. Feed this entire document as context on every phase prompt — it comfortably fits the 1M-token context window — and append the actual output/decisions from the prior phase before starting the next one, so later phases are grounded in what was really built, not just what this document originally planned.

Phases

Field	Type	Description
Phase 1	Data layer	Schema, migrations, ingestion for all sources in Section 4, extraction, fact-chain storage
Phase 2	Correlation + causal engine	Section 5.4 / 5.5 / 9, built and tested against real data now sitting in the DB from Phase 1
Phase 3	API layer	Section 8 routes serving Phase 1/2 output, WebSocket feed
Phase 4	Tier 1 graphics	Built against the synthetic dataset (Section 12), independent of Phases 1-3 timing
Phase 5	Wire graphics to real API	Swap synthetic data source for the real API from Phase 3; purge _synthetic rows
Phase 6	Remaining features + Tiers 2/3	Instability index, bias view, multi-language, resilience hardening, lighter graphics tiers

SECTION 16 // COMPLETION CRITERIA

v1 done checklist

- LICENSE (AGPL-3.0) and .gitignore (Python + Node additions, Section 14) committed before the first application code commit.
- All sources in Section 4 successfully ingesting on their configured interval with visible health status.
- Fact chain accumulating in extracted_facts with working embeddings.
- Correlation engine producing story clusters from real data, with both same-window and historical-chain links observed at least once.
- Causal storylines generating with confidence levels, cached correctly, using the exact prompt in Section 9.
- Instability index computing on schedule and trending on a visible chart.
- Live feed updating in real time via WebSocket, with REST polling fallback verified.
- Map rendering real events at Tier 1 on the owner's machine at target frame rate (Section 11.3).
- Every rendered story / event / fact traceable to a visible, linked source (Section 6.8 enforcement verified, not just assumed).
- Bias/blindspot view showing at least the curated outlet set from Section 4.1.
- System survives a single source going down without crashing or blocking the rest of the pipeline (Section 10.2 policy verified with a deliberate test).
- All _synthetic-flagged rows purged after Phase 5 wiring is confirmed working.

SECTION 17 // GLOSSARY

Glossary

Event — A single normalized occurrence extracted from one raw item — one article, one earthquake reading, one price move.

Extracted fact — A structured who/what/where/when record pulled from an event or article, stored permanently in the fact chain.

Fact chain — The permanent, growing collection of all extracted facts, used for correlation against events that happened at any point in the past, not just recently.

Story — A cluster of correlated events/facts, presented as a single unified item with a causal narrative.

Same-window correlation — Matching events that occurred close together in time (within the configured gap threshold).

Historical correlation — Matching a new event against the entire historical fact chain with no time limit, using a stricter similarity bar.

Instability index — A single 0-100 composite score reflecting global event volume, severity, and geographic spread over a rolling window.

Tier — One of three graphics rendering modes (Tier 1 full / Tier 2 standard / Tier 3 light), selected by device capability.

Confidence — A high/medium/low label attached to every AI-generated causal narrative, reflecting how well the evidence supports the stated cause-and-effect claim.

AGPL-3.0 — GNU Affero General Public License v3.0 — the project's license (Section 14.1). Copyleft with a network-use clause: modified versions run as a hosted service must still make their source available.

APPENDIX A // FORWARD-LOOKING NOTES

Future public / multi-user architecture

This section exists only so v1 is not built in a way that makes a future transition harder than necessary. None of this is in scope for the current build — it is a forward-looking note on transferability.

A.1 What would need to change

- **Authentication & multi-tenancy** — v1 has none. A public version needs accounts, session handling, and per-user preference storage (saved filters, followed regions, etc.).
- **Database hosting** — native local Postgres becomes a managed cloud Postgres instance (RDS, Cloud SQL, or similar) with connection pooling (pgbouncer) for concurrent load.
- **Ingestion scaling** — APScheduler-in-process is fine for one user; a public version with many concurrent readers still ingests centrally (ingestion volume doesn't scale with users), but the job runner should move to a proper task queue (Celery + Redis, or a managed equivalent) for reliability and observability.
- **Causal-link LLM cost** — becomes the dominant cost driver at scale. Aggressive caching (already required in v1) becomes load-bearing; consider a self-hosted local model (Ollama) as a cost-reduction path for high-volume public deployments.
- **Graphics tiers** — Section 11 was deliberately built to already support this transition: Tiers 2 and 3 exist specifically for lower-end devices, which matters far more once the audience isn't limited to one known machine.
- **Rate limiting & abuse protection** — irrelevant for single-user local use, essential the moment the API is public.
- **Hosting** — move from native Windows processes to containerized deployment (Docker) for portability and reproducibility. Skipping Docker in v1 was purely to reduce setup friction for one developer on one machine, not a rejection of containers generally.

A.2 What would NOT need to change

- The core schema in Section 6 — already tenant-agnostic; adding a nullable owner reference where needed is additive, not a rewrite.
- The correlation and causal-linking logic in Sections 5.4-5.5 and Section 9 — operates on global data, not per-user data, so it is inherently already shared-infrastructure shaped.
- The fact chain concept itself — more users generates more ingestion diversity, which strengthens correlation quality over time rather than complicating it.

// END OF DOCUMENT — TALKDIPLOMACY LIVE v1 BUILD MANUAL — REV 1.2